Executive Report Summary: Electricity Meter Anomaly Detection

EDAB6808 Assignment 2
Buhle Mbokodo (2024007920)
Reatile Seekoei (2021109463)

Key Findings:

This report evaluates four methods for detecting anomalous hourly electricity readings across buildings. Historical consumption features and contextual variables were combined with building-aware validation, using separate building groups for hyperparameter tuning, threshold selection and evaluation. XGBoost achieved the highest Average Precision of 0.7540, with a precision of 95.36% and recall of 62.00%. Ten correctly detected anomalies were examined through comparisons with earlier readings. The final model flagged 27,252 of 1,800,567 unseen test readings are anomalous. The findings support XGBoost for prioritising readings for review, while highlighting missed anomalies, variation between buildings and the absence of labelled test results.

1 Executive summary

Recommendation. Use XGBoost as the preferred model for the supplied electricity anomaly-detection task. It achieved the highest pooled Average Precision (AP), **0.7540**, among the four compared methods. At the fixed threshold of **0.864411**, its evaluation precision was **95.36%**, recall **62.00%**, and F1 **0.7514**. This supports a selective review process: most flagged readings were labelled anomalies, but approximately 38% of actual evaluation anomalies were missed.

The objective was to identify unusual hourly electricity readings and generate a binary prediction for every reading from 206 unseen test buildings. The training file contained 1,749,494 readings from 200 buildings, with 37,296 anomalies (2.13%). A majority-class model would therefore appear accurate while detecting no anomalies. The analysis focused on AP and the precision–recall trade-off, rather than accuracy alone.

The pipeline retained the original class imbalance, distinguished missing readings from recorded values, and used each building's previous readings to provide context. Optional building and weather variables were merged using building ID and timestamp. Historical features used elapsed time, which is important when hourly rows are absent.

Hyperparameter tuning, threshold selection and evaluation used disjoint pools of 90, 33 and 77 buildings, respectively. Five-fold evaluation excluded each held-out building from the corresponding model fit. Random Forest, XGBoost, Logistic Regression and a label-assisted Isolation Forest baseline were compared. Random Forest generated fewer false alarms than XGBoost but missed more anomalies; the other two methods performed substantially worse.

The selected model was refitted on a stratified sample of 400,000 training readings. It scored all 1,800,567 test readings and flagged **27,252 (1.51%)**. Predictions were joined to their original row IDs, avoiding misalignment introduced by sorting during feature engineering. The test anomaly percentage describes predictions and is not test accuracy.

Decision implications

- High precision can reduce unnecessary investigations, but recall remains insufficient for an application that requires detecting nearly every anomaly.
- Ten selected examples show sharp drops to a recorded value of 1.0; historical comparisons explain their context without proving an equipment fault.
- Performance varied across building groups. The winning model was selected using the evaluation results, and unseen-test labels were unavailable.

2 Data understanding and quality control

The supplied files describe hourly electricity consumption during 2016. `train.csv` contains building ID, timestamp, meter reading and the binary anomaly label; `test.csv` replaces the label with a submission row ID. The assignment identifies the consumption unit as kWh.

Table 1: Audit of the supplied main data files.

Check	Training data	Test data
Readings	1,749,494	1,800,567
Buildings	200	206
Labelled anomalies	37,296 (2.13%)	Not supplied
Missing meter readings	107,653 (6.15%)	94,864 (5.27%)
Duplicate building–timestamp keys	0	0
Negative recorded readings	0	0
Timestamp range	1 January–31 December 2016	
Shared train/test building IDs	0	

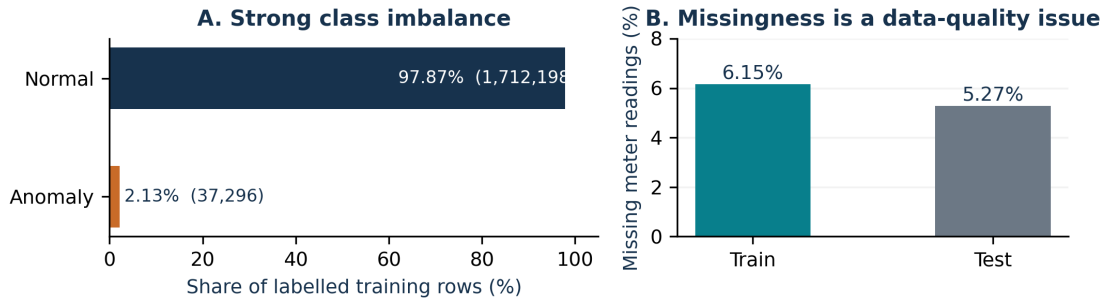


Figure 1: Training-label prevalence and missing meter readings. Missingness and anomalous usage are separate concepts. Source: supplied raw data; figures regenerated for this report.

Cleaning decisions

Timestamps were parsed as datetimes and readings sorted by building and time. No duplicate keys or invalid timestamps required removal. Missing readings were retained, with a missingness indicator and training-fitted median imputation for numerical model inputs. Recorded zero values were retained. Neither a missing value nor a zero was automatically relabelled as anomalous. All missing training meter readings had label 0; this is a descriptive finding, not a rule for assigning labels to future data.

The audit found 1,295 gaps longer than one hour between consecutive observed training readings. A shift by 24 rows can therefore differ from a lag of 24 hours. Historical features were corrected to use elapsed time. An absent timestamp row is also different from a row containing a missing meter value.

Optional contextual data

The optional files were left-joined on building ID and timestamp with one-to-one validation. The executed merge checks showed no unmatched main rows. A whitelist retained building metadata, weather measurements and the holiday indicator, excluding any label-bearing fields. These variables provide context for usage levels; their separate contribution was not measured by an ablation study.

3 Exploratory findings

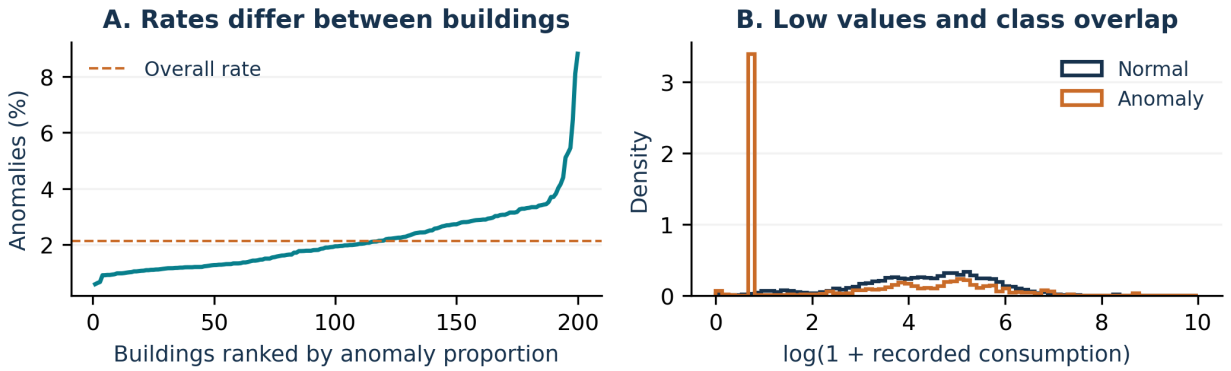


Figure 2: Building anomaly proportions and class-conditional consumption distributions. Distribution curves exclude missing readings and use a log transformation for display. The distribution panel shows the range $0 \leq \log(1+x) \leq 10$.

Buildings differ. Training-building anomaly proportions ranged from 0.58% to 8.82%, with a median of 1.94%. This variation motivates building-aware validation and historical comparisons within each building, rather than a single universal consumption cutoff.

Anomalies are not necessarily high readings. Among recorded values, normal readings had mean 181.08 kWh and median 86.00 kWh; anomalous readings had mean 129.34 kWh and median 10.00 kWh. Lower anomalous medians indicate that unusually low consumption is relevant. The overlapping distributions also show that consumption magnitude alone cannot reliably separate classes.

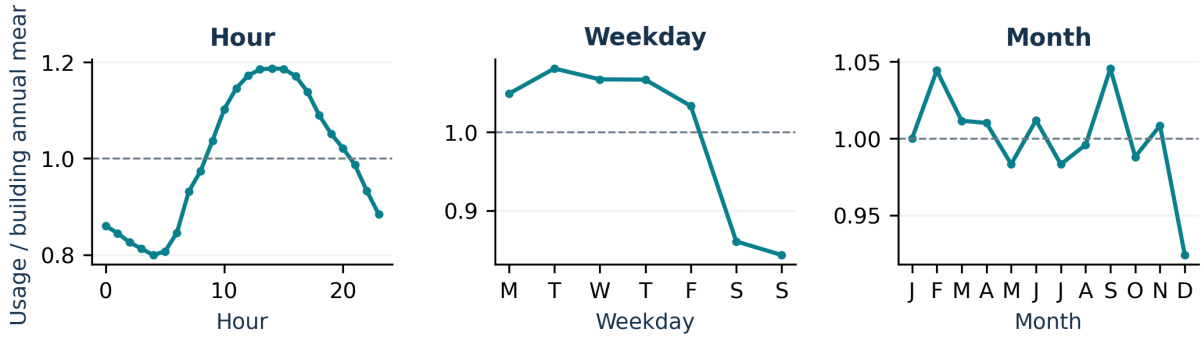


Figure 3: Descriptive hourly, weekday and monthly profiles. Each reading is divided by its building's annual mean; period means are calculated within buildings and then averaged equally across buildings. The dashed line represents the building-normalised baseline.

Time provides context. Consumption was higher around midday and lower in early morning. Weekend averages were below weekday levels. Monthly variation was present, with February and September above the normalised baseline and December below it. These observations support calendar variables and daily/weekly historical windows.

Only one year was supplied, so monthly differences do not establish a repeatable annual seasonal pattern. Annual-mean normalisation is used here only to describe the data: it is *not* a predictive feature, because an entire-year baseline would incorporate future readings at earlier timestamps.

4 Feature engineering and candidate models

Historical and contextual representation

The final input matrix had 41 columns before categorical encoding: 40 numerical variables and the categorical `primary_use` variable. The raw reading was represented through a log transformation and contextual deviations. Building ID, timestamp, row ID and anomaly label were excluded as direct predictors.

Table 2: Feature families and their analytical roles.

Feature family	Construction and role
Current observation	$\log(1+x_t)$ and a missingness indicator distinguish recorded magnitude from unavailable data.
Calendar	Hour, weekday, month, day of year, weekend status and cyclic encodings describe recurring time patterns.
Exact-time lags	Building-specific readings at $t-1$, $t-24$ and $t-168$ hours; unavailable matches remain missing until preprocessing.
Rolling history	Mean, standard deviation and median over the preceding 24 and 168 hours, excluding t; at least six nonmissing readings required.
Relative deviations	Differences from lags, ratios to rolling medians, and conventional mean/std z-scores measure changes from recent context.
Optional context	Building use, size, age, floors, site, holiday status and weather measurements contextualise consumption.

For a window w, the standard deviation score was

$$z_{t,w} = \frac{x_t - \mu_{t,w}}{\sigma_{t,w} + 10^{-6}}, \qquad r_{t,w} = \frac{x_t}{\widetilde{x}_{t,w} + 10^{-6}}.$$

Here μ, σ and $\widetilde{x}$ use only preceding readings from the same building. The z-score is conventional, not a median- or MAD-robust score. Infinite engineered values were replaced with missing values. Near-zero Baselines can still produce large finite ratios, so ratios require context.

Numerical inputs were median-imputed; categorical inputs used the most frequent category and one-hot encoding with unknown categories handled. Each model fit used a cloned preprocessing pipeline fitted on its own training observations. Logistic Regression additionally used `StandardScaler(with_mean=False)`.

Four complementary methods

Random Forest averages tree predictions and captures nonlinear relationships; the selected setup used 187 trees, depth 20, minimum leaf size 15, and square-root feature selection, with balanced subsample class weights. **XGBoost** uses boosted trees; its selected setup included 281 trees, depth 10 and learning rate approximately 0.1207. **Logistic Regression** supplied a linear baseline with $C = 1$, balanced class weights and a 1,000-iteration limit. **Isolation Forest** isolates unusual observations; This implementation used 250 trees and up to 100,000 labelled normal training readings. It is therefore a label-assisted baseline, not a fully unsupervised experiment. Negated `score_samples` made larger scores indicate greater anomaly evidence.

5 Validation, tuning and evaluation measures



Figure 4: Disjoint pools for model development. In evaluation, the held-out 15 or 16 buildings are excluded; eligible training data come from all other 184 or 185 buildings, including the tuning and threshold-selection pools.

Sequence of modelling decisions

1. **Tune on 90 buildings.** A stratified sample of up to 120,000 rows was split by building into 67 training and 23 validation buildings. Optuna's seeded TPE sampler ran 12 trials each for Random Forest and XGBoost, maximising AP. Their best development AP scores were 0.7818 and 0.8085. These are tuning results, not final performance estimates.
2. **Choose cutoffs on 33 separate buildings.** Models fitted on the 90-building pool scored this separate pool. Each cutoff maximised F1 on its precision–recall thresholds, after which it was fixed.
3. **Evaluate on 77 separate buildings.** Five-fold GroupKFold kept buildings intact. Each fold held out 15 or 16 evaluation buildings; models fitted on stratified samples of up to 400,000 eligible training rows. Every evaluation row received one held-out prediction.

Table 3: Fixed cutoffs selected before the five-fold evaluation.

Model	Cutoff	Model	Cutoff
XGBoost	0.864411	Random Forest	0.855725
Logistic Regression	0.808623	Isolation Forest	0.480918

Measures suited to rare anomalies

$$P = \frac{TP}{TP+FP}, \qquad R = \frac{TP}{TP+FN}, \qquad F_1 = \frac{2PR}{P+R}.$$

Precision measures the reliability of alerts; recall measures anomaly coverage. F1 summarises their trade-off. AP evaluates score ranking over thresholds: $\mathrm{AP} = \sum_n (R_n - R_{n-1}) P_n$. The reported implementation is noninterpolated and differs from trapezoidal PR-AUC. ROC-AUC and confusion counts provide complementary information. With only 2.13% training anomalies, an all-normal prediction would have 97.87% accuracy but zero recall, illustrating why accuracy can be misleading.

The design separates tuning and cutoff selection from the held-out evaluation buildings. However, the evaluation comparison subsequently selects the winner, so it is not a completely independent post-selection test. Historical features are time-causal, but validation targets unseen buildings within the supplied year, rather than forecasting a future year.

6 Model comparison and interpretation

Table 4: Pooled out-of-fold results on 673,433 evaluation readings, including 13,514 anomalies (2.01%). All cutoffs were fixed beforehand.

Model	AP	ROC-AUC	Precision	Recall	F1
XGBoost	.7540	.9298	.9536	.6200	.7514
Random Forest	.7417	.9413	.9963	.5029	.6684
Logistic Regression	.1083	.7864	.1885	.4158	.2594
Isolation Forest	.0433	.6542	.0712	.0701	.0706

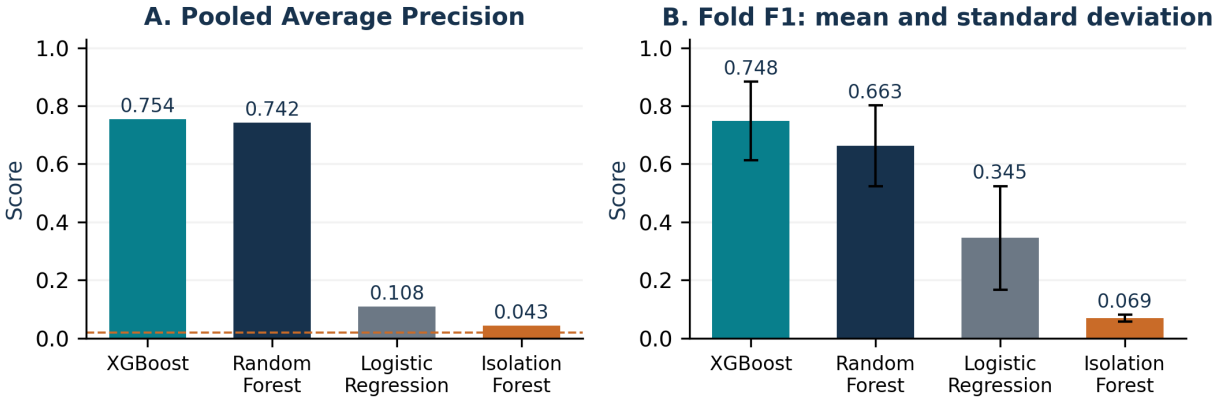


Figure 5: Pooled AP and fold-level F1. The dashed AP line shows evaluation anomaly prevalence as a no-skill reference. Error bars show the sample standard deviation across five folds, not a confidence interval. Model metrics are reconstructed from verified Colab outputs.

XGBoost ranked first on AP and F1. It correctly detected 8,378 anomalies, with 408 false alarms and 5,136 missed anomalies. Random Forest was more selective: 6,796 detected anomalies, 25 false alarms and 6,718 missed anomalies. Its higher ROC-AUC does not change the AP-based selection rule. Logistic Regression and Isolation Forest generated many more false alarms while providing weaker anomaly ranking.

Table 5: Unweighted means and sample standard deviations across five folds. Values were rounded to three decimals in the executed summary.

Model	AP: mean ± std	Recall: mean ± std	F1: mean ± std
XGBoost	.758 ± .130	.627 ± .160	.748 ± .135
Random Forest	.745 ± .136	.508 ± .141	.663 ± .139
Logistic Regression	.331 ± .190	.415 ± .123	.345 ± .179
Isolation Forest	.051 ± .013	.067 ± .041	.069 ± .011

Strong average performance is not uniform performance. XGBoost's recall standard deviation of 0.160 indicates material variation between building groups. A low standard deviation for Isolation Forest instead reflects consistently weak performance.

Pooled metrics combine all held-out predictions, whereas fold means average five separate evaluations. AP also depends on the ordering of scores across separately fitted models. Logistic Regression's substantial pooled/fold AP gap warrants investigation of score comparability; it should not be explained solely by fold-size weighting. No significance test establishes that XGBoost's small AP advantage over Random Forest is statistically decisive.

7 Interpretation of ten point anomalies

Ten high-scoring, correctly detected anomalies were selected from distinct evaluation buildings. Every example has actual label 1, predicted label 1, and an XGBoost score approximately 1.000, above the fixed cutoff. Table 6 compares each recorded value with the same hour one day earlier.

Table 6: Ten selected point anomalies; consumption values are in kWh. Drop $= 100(1 - x_t / x_{t-24h})$. Each reading is 1.00, rather than missing. The median uses the preceding 24 hours.

No.	Building	Timestamp (2016)	Prior median	24h earlier	Drop (%)
1	111	30 Sep, 23:00	1.00	355.00	99.72
2	1253	18 Jul, 19:00	306.92	306.92	99.67
3	1275	18 Jul, 21:00	103.28	103.28	99.03
4	1259	01 Jul, 17:00	1.00	155.00	99.35
5	990	30 Mar, 08:00	107.50	123.00	99.19
6	961	30 Mar, 08:00	326.00	316.00	99.68
7	1323	25 Aug, 17:00	1.00	230.00	99.57
8	1252	14 Jan, 16:00	451.99	520.00	99.81
9	147	05 Dec, 09:00	152.62	148.00	99.32
10	886	25 Jan, 23:00	1.00	164.00	99.39

Examples 2, 3, 5, 6, 8 and 9 are substantially below both their preceding 24-hour median and the same-hour daily lag. For example, building 1252 recorded 1.00 compared with a historical median of 451.99 and a daily lag of 520.00, a 99.81% daily decline. Building 961 similarly fell from 316.00 to 1.00 despite a historical median of 326.00.

Examples 1, 4, 7 and 10 have a historical median already equal to 1.00. Their immediate median therefore does not explain the flag. However, the daily lags of 355.00, 155.00, 230.00 and 164.00 show much higher usage one day earlier. This is consistent with a low-reading episode that has already influenced the short rolling baseline, rather than an isolated spike.

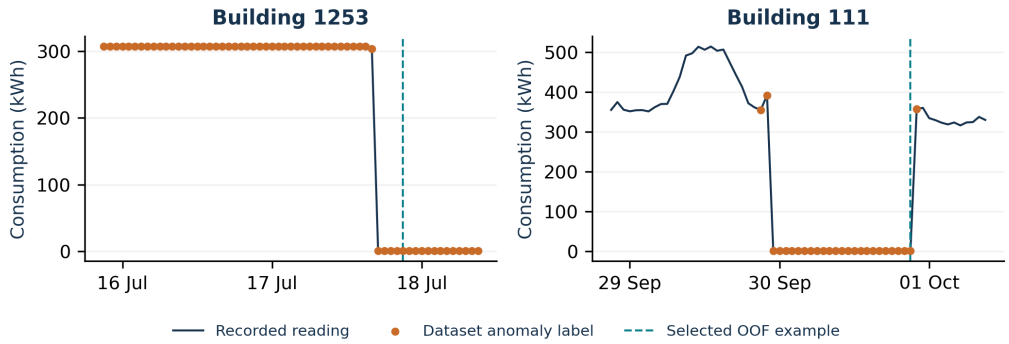


Figure 6: Original labelled training readings around two selected held-out examples. Orange markers indicate supplied anomaly labels; the dashed line identifies the point examined in the report. These markers are not additional model predictions.

These comparisons are contextual evidence, not causal diagnoses or exact feature attributions for XGBoost. A shutdown, meter issue or operational change cannot be confirmed from these columns alone. Selecting the strongest scores favoured a repeated value of 1.00, so the examples are not representative of all detected anomaly types. Consecutive labelled readings may form an episode; the assignment's point examples do not imply ten independent incidents.

8 Final predictions, limitations and conclusion

Final fit and submission integrity

After model selection, XGBoost was fitted on a stratified sample of 400,000 training readings with approximately 2.13% anomalies. This is a computational sample, not a fit using every training row or every labelled anomaly. All test rows were scored in batches of 100,000 and classified using the unchanged threshold of 0.864411. The model flagged **27,252** of **1,800,567** readings: **1.51%**; the remaining 1,773,315 readings were classified as normal.

Feature engineering sorts by building and timestamp. Predictions were therefore attached to `test_fe.row_id`, then joined into `sample_submission.csv` order. Structural checks confirmed unique IDs, complete predictions, binary values, matching row counts and the exact `row_id, anomaly` schema. The generated output was `anomaly_submission.csv`.

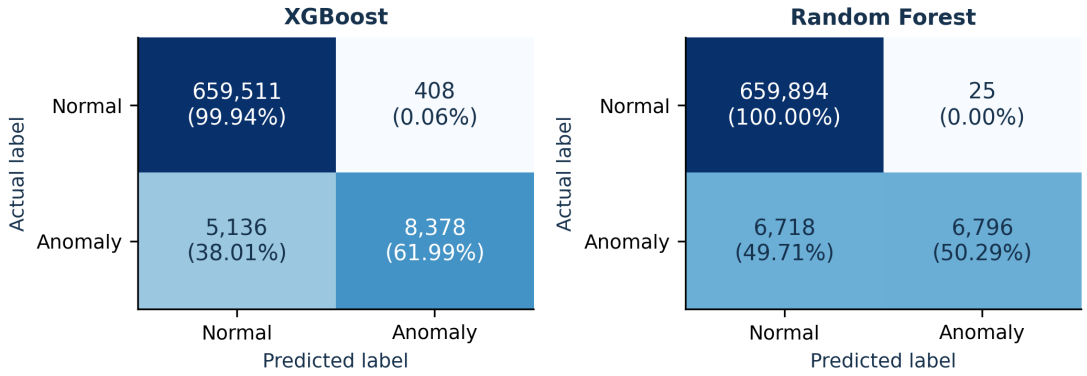


Figure 7: Evaluation confusion matrices for the two leading models: exact counts and percentages within each actual class. These are validation results, not unseen-test results.

Operational interpretation and limitations

XGBoost provides useful prioritisation for manual review, with broader anomaly coverage than Random Forest at the chosen cutoffs. False alarms and missed anomalies have different consequences; F1-based threshold selection does not encode investigation cost, outage severity or a service-level target. Such business costs were not supplied, so no monetary savings or production-readiness claim is made.

Unseen test accuracy cannot be measured without labels. The evaluation results selected the winning method, leaving some model-selection optimism. Building-group variation and changes in score distributions after refitting also limit confidence in transferring one fixed cutoff. The one-year data window does not establish future-year stability. Optional context was not evaluated through feature ablation; model convergence, probability calibration and performance by individual anomaly type were not separately established.

Conclusion

The pipeline satisfies the central anomaly-detection tasks: audited data, time-causal features, four compared methods, held-out-building evaluation, ten contextualised point anomalies, and correctly aligned binary test predictions. XGBoost was the strongest model under the specified AP-based selection rule. Its high precision should be interpreted alongside its 62.00% recall and fold variation. The final CSV is a structurally validated prediction deliverable; its correctness against unseen test labels remains unmeasured.

References

Breiman, L. (2001). Random forests. *Machine Learning*, 45:5–32.

Chen, T. and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794.

Liu, F. T., Ting, K. M., and Zhou, Z.-H. (2008). Isolation forest. In *2008 Eighth IEEE International Conference on Data Mining*, pages 413–422.

Provost, F. and Fawcett, T. (2013). *Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking*. O'Reilly Media, Sebastopol, CA.

scikit-learn developers (2026a). average_precision_score: Api documentation. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.average_precision_score.html. Accessed 4 October 2026; documentation access date does not specify the Colab runtime version.

scikit-learn developers (2026b). GroupKFold: Api documentation. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GroupKFold.html. Accessed 4 October 2026.